

LUMI: Unsupervised Intent Clustering with Multiple Pseudo-Labels

I-Fan Lin
Leiden University
Leiden, The Netherlands
i.lin@liacs.leidenuniv.nl

Faegheh Hasibi
Radboud University
Nijmegen, The Netherlands
faegheh.hasibi@ru.nl

Suzan Verberne
Leiden University
Leiden, The Netherlands
s.verberne@liacs.leidenuniv.nl

Abstract

In this paper, we propose an intuitive, training-free and label-free method for intent clustering in conversational search. Current approaches to short text clustering use LLM-generated pseudo-labels to enrich text representations or to identify similar text pairs for pooling. The limitations are: (1) each text is assigned only a single label, and refining representations toward a single label can be unstable; (2) text-level similarity is treated as a binary selection, which fails to account for continuous degrees of similarity. Our method LUMI is designed to amplify similarities between texts by using shared pseudo-labels. We first generate pseudo-labels for each text and collect them into a pseudo-label set. Next, we compute the mean of the pseudo-label embeddings and pool it with the text embedding. Finally, we perform text-level pooling: Each text representation is pooled with its similar pairs, where similarity is determined by the degree of shared labels. Our evaluation on four benchmark sets shows that our approach achieves competitive results, better than recent state-of-the-art baselines, while avoiding the need to estimate the number of clusters during embedding refinement, as is required by most methods. Our findings indicate that LUMI can effectively be applied in unsupervised short-text clustering scenarios. Our source code is available at https://anonymous.4open.science/r/pseudo_label-7AE1/README.md

CCS Concepts

• **Computing methodologies** → **Unsupervised learning**; • **Information systems** → **Document representation**; **Information retrieval query processing**; **Clustering and classification**.

Keywords

Intent clustering, unsupervised clustering

1 Introduction

Intent clustering groups unlabeled user utterances in conversational systems into clusters of similar intents. This facilitates query intent identification [2, 3, 9], enabling the development of intent-aware information-seeking systems [11, 18] and customer-facing conversational agents [15].

Many studies have addressed this task, mostly aiming at fine-tuning an embedder to learn representations for clustering [14, 27]. These contrastive learning approaches generally require labeled training data, and their loss functions tend to be complex, making hyperparameter optimization difficult. LLM-guided label-free approaches have been proposed to address these challenges, either guiding representation refinement using LLM-generated pseudo-labels [5, 24], or by performing similar-text pooling based on LLM judgments [16]. The LLM-guided approach performs on par with

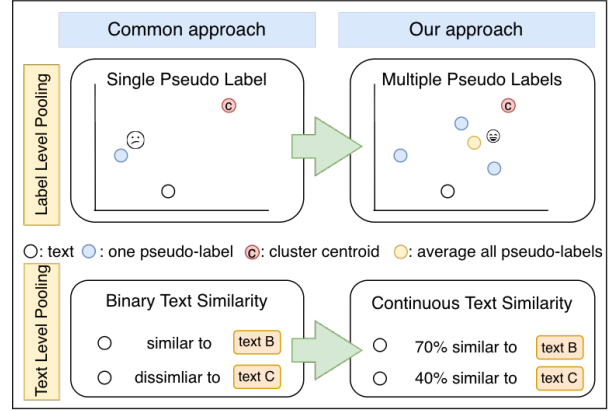


Figure 1: Illustration of our proposed method. Label-level pooling: pooling with multiple pseudo-labels. Text-level pooling: similar pairs are obtained via soft selection based on shared pseudo-labels.

contrastive learning but with two main limitations: (1) noisy pseudo-labels due to single-shot labeling, and (2) selection restricted to binary decisions, which ignores continuous similarity.

To address these limitations, we propose a method, LUMI, that **constructs a pseudo-label set to enhance text representations**. Figure 1 shows the general idea of our approach. Each text is assigned multiple temporary, LLM-generated labels, which serve as cluster-level signals. These pseudo-labels are used in two ways: first, their embeddings are averaged to obtain a stable representation (label-level pooling), and second, they define similarity between texts for further pooling with similar texts (text-level pooling).

Our paper makes three contributions: (1) we theoretically justify why multi-labeling improves clustering, (2) we empirically show that LUMI outperforms state-of-the-art baselines and remains stable across different LLMs and embedders, and (3) our approach enables training-free, and label-free embedding refinement for effective intent clustering, without requiring the number of clusters during refinement.

2 Related Work

Several approaches have explored the use of LLMs to improve representations for short text clustering. We categorize them into two types: LLM-as-a-matcher and LLM-as-a-labeler: In the **LLM-as-a-matcher** setting, the LLM is used only to select similar candidates. Zhang et al. [26] use LLMs to select quality triplets for fine-tuning an embedder. Feng et al. [6] use LLMs to re-cluster edgepoints. Lin et al. [16] first encode all texts using an existing embedder and

then use an LLM to select similar candidate texts for each instance; the embeddings of these candidates are average-pooled to produce the final representation for clustering. In contrast, the **LLM-as-a-labeler** approach directly uses LLMs to generate (pseudo-)labels for texts. De Raedt et al. [5] first select prototype texts and use an LLM to generate their labels. They then use the LLM again to label/generate the remaining texts with these prototypes. The enriched texts are subsequently used to improve representations. Viswanathan et al. [25] use LLMs to generate key phrases for each text, which are then separately encoded into embeddings and concatenated for clustering. Unlike previous work, LUMI is a hybrid method that bridges LLM-as-a-matcher and LLM-as-a-labeler via multiple pseudo-labeling.

3 Theoretical Foundations

We provide a theoretical analysis of the label-level pooling here, noting that it constitutes only one component of our overall method.

3.1 Problem Formulation

Following established formulations of emerging intent discovery [16, 24, 26] we assume a collection of text data, denoted as $D = \{x_i\}_{i=1}^N$, where each data point $x_i \in D$ corresponds to a short text, and N is the total number of data points. The task is to partition D into K cluster sets, denoted as $\{S_l\}_{l=1}^K$. Our approach does not rely on a pre-defined K to refine the representation of data points. This is unlike previous studies that involve known K in their method [5, 6, 26].

3.2 Theoretical Justification

In this section, we provide a theoretical justification for why label-level pooling improves clustering performance. We adopt the standard clustering assumption that a cluster S is good if its points are tightly concentrated, i.e., the distances between the individual data points and the centroid of the cluster are small.

For an arbitrary cluster, Let x be a text from the cluster S and $\vec{X} = [X_1, \dots, X_d]$ denote its embedding, where each entry X_h is the h -th dimension of the text embedding. Let $\vec{Y}_j = [Y_{j1}, \dots, Y_{jd}]$, for $j = 1, \dots, M$, denote the embeddings of M pseudo-labels of the text, where Y_{jh} is the h -th dimension of the j -th pseudo-label embedding. Let c_h be the h -th dimension of the cluster centroid of the text.

It has been shown empirically that the use of one pseudo-label improves unsupervised clustering [5, 24]. We theoretically prove that the average pseudo-label embeddings $\bar{Y} := \frac{1}{M} \sum_{j=1}^M \vec{Y}_j$ are closer to the cluster centroid than individual pseudo-label embedding $\vec{Y}_j$, which will lead to a better clustering result. Formally, the goal is to ensure the following inequality holds:

$$\sum_{h=1}^d E[(\bar{Y}_h - c_h)^2] < \sum_{h=1}^d E[(Y_{jh} - c_h)^2] \quad (1)$$

where d is the embedding dimension. For notational simplicity, we consider h as a representative dimension and omit the dimension index h from Eq. (1), rewriting it as:

$$E[(\bar{Y} - c)^2] < E[(Y_j - c)^2]. \quad (2)$$

Note that if Eq. (2) holds, meaning that each dimension of $\bar{Y}$ has a lower mean squared error than the corresponding dimension in $\vec{Y}_j$,

Algorithm 1 Derivation of Averaged Pseudo-label Embeddings

Require: D, D_l , embedder f , candidate size b , LLM

Ensure: Averaged Pseudo-Label Embeddings of the dataset $D \mathbf{Z} \in \mathbb{R}^{N \times d}$

```

1: Initialize Averaged Pseudo-Label Embeddings:  $\mathbf{Z} \leftarrow \mathbf{0} \in \mathbb{R}^{N \times d}$ 
2: Initialize pooled embeddings:  $\mathbf{P} \leftarrow \mathbf{0} \in \mathbb{R}^{N \times d}$ 
3: for  $x_i \in D, l_i \in D_l$  do
4:    $\mathbf{p}_i \leftarrow \text{Normalize}\left(\frac{f(x_i) + f(l_i)}{2}\right)$  {# Text-label embedding}
5:    $\mathbf{P}[i] \leftarrow \mathbf{p}_i$ 
6: end for
7: for  $x_i \in D, l_i \in D_l$  do
8:    $M_i \leftarrow \{(x_{i1}, l_{i1}), \dots, (x_{ib}, l_{ib})\}$  {# Top- $b$  closest texts with a different initial pseudo-label using embeddings  $\mathbf{P}$ }
9:    $O_i \leftarrow \text{LLM}(x_i, M_i)$  {# Performing Multi-label classification to get LLM selected pseudo-label set for text  $x_i$ .  $0 \leq |O_i| \leq b$ }
10:   $L_i \leftarrow \{l_i\} \cup O_i$  {# Final pseudo-label set for  $x_i$ }
11:   $\mathbf{z}_i \leftarrow \text{Normalize}\left(\frac{1}{|L_i|} \sum_{l \in L_i} f(l)\right)$ 
12:   $\mathbf{Z}[i] \leftarrow \mathbf{z}_i$  {# text  $x_i$  Averaged Pseudo-Label Embedding}
13: end for
14: return  $\mathbf{Z}$ 
```

then Eq. (1) holds. The inequality 2 is equivalent to:

$$\text{Var}(\bar{Y}) + \text{Bias}(\bar{Y}, c)^2 < \text{Var}(Y_j) + \text{Bias}(Y_j, c)^2. \quad (3)$$

To prove Eq. (3), we make the following assumptions about all pseudo-labels generated for the text x :

Assumption 1. The random variable Y_j , representing a dimension of the a pseudo-label embedding for text x , is decomposed as:

$$Y_j = c + \alpha U + \epsilon_j, \quad (4)$$

where $U := X - c$ is the text-specific deviation, $\alpha \in \mathbb{R}$ controls its contribution, and $\epsilon_j \in \mathbb{R}$ is random noise from the label generation process. This assumption also implies that the LLM-generated pseudo-labels for a randomly selected text x are derived from the same input x (for analysis simplicity).

Assumption 2. The i.i.d. variables $\{\epsilon_j\}_{j=1}^M$ are independent of U .

This assumption implies that variables $\{\epsilon_j\}_{j=1}^M$, driven from the same distribution, have the same variance σ_ϵ^2 . Under the assumption 1, $\text{Bias}(\bar{Y}, c)^2 = \text{Bias}(Y_j, c)^2$ in Inequality (3). Therefore, it remains only to show $\text{Var}(\bar{Y}) < \text{Var}(Y_j)$. We write the variance of Y_j and $\bar{Y}$ as:

$$\text{Var}(Y_j) = \alpha^2 \sigma_U^2 + \sigma_\epsilon^2, \quad (5)$$

$$\text{Var}(\bar{Y}) = \alpha^2 \sigma_U^2 + \frac{1}{M} \sigma_\epsilon^2, \quad (6)$$

where σ_U^2 is variance of U . Based on Eqs. (5) and (6), inequality (2) and therefore inequality (1) hold. This shows that averaging multiple pseudo-labels reduces noise from the label generation process. Empirically, pseudo-label embeddings also tend to reduce text-specific variability (α), which further improves clustering, though this effect is not assumed in the analysis. Note that in LUMI, pseudo-labels for a given x can be derived from other x 's, which often improves label quality.

4 Computational Method: LUMI

Our method has three main steps: (1) generating a single initial pseudo-label for each text, (2) performing multi-label classification to assign multiple pseudo-labels to each text; and (3) using the resulting pseudo-labels for label-level and text-level pooling to obtain a refined embedding for each text.

4.1 Generation of the pseudo-labels with LLM

For each text $x_i \in D$, we derive a single pseudo-label by prompting an LLM. To ensure labels reflect cluster-level semantics, we provide the LLM with the top b closest texts from the collection based on cosine similarity, where the embeddings are obtained from an existing pre-trained embedder. In our experiments, we set candidate size $b = 10$, since larger b will increase the LLM inference cost. Because the pre-trained embedder is not optimized for our dataset, we apply a simple adaptation: we prompt the LLM to generate labels based only on neighbors that closely match the text. We denote l_i as the initial pseudo-label of x_i , and $D_l = [l_1, \dots, l_N]$ be the ordered list of pseudo-labels corresponding to the dataset D .

4.2 Label-level Pooling

To get pseudo-labels, we perform multi-label classification for each text. For a text $x \in D$, we provide the LLM with the top- b ($b = 10$) nearest neighbors, each associated with a different pseudo-label. The embeddings for selecting these neighbors are computed by pooling the text with its initial pseudo-label, a strategy shown to improve embeddings, even when only a single pseudo-label is used [5, 24]. Once the LLM assigns new pseudo-labels, we encode them separately and pool them together with the initial label to form the averaged pseudo-label embedding, denoted as z . Algorithm 1 details how z (uppercase Z for all texts in D) is computed. We then follow prior work [5] and compute the average of the text embedding $f(x)$ and z , denoted as $z' = \frac{f(x) + z}{2}$ (uppercase Z' for all texts in D)

4.3 Text-level Pooling

Once we obtain the embeddings Z' , which incorporate the pseudo-label level information, we use them to perform text-level pooling. Lin et al. [16] have conceptually shown that pooling with similar texts shrinks clustering variance and highlight the importance of identifying similar text pairs. However, existing approaches [5, 16] treat similarity as binary, without accounting for the varying degrees of similarity; we address this by using a continuous similarity-based pooling. Motivated by the intuition that texts sharing more pseudo-labels should be considered more similar, we propose to use the Jaccard similarity to weight similar texts according to the extent of their shared labels when performing pooling over Z' .

$$w_{ij} = \frac{|L_i \cap L_j|}{|L_i \cup L_j|},$$

where, L_i, L_j denote the updated sets of pseudo-labels x_i and x_j after multi-label classification. Then, we update:

$$\tilde{z}'_i = \frac{\sum_{x_j \in D} w_{ij} z'_j}{\sum_{x_j \in D} w_{ij}},$$

where, z'_j is the embedding of x_j after averaging the text embedding x_j with its averaged pseudo-label embedding z_j and $w_{ii} = 1$. The refined embedding $\tilde{Z}$ will be used to form clusters.

5 Experiments and Results

5.1 Experimental Setup

Datasets and models Following most prior work [6, 16, 22, 24, 26], we use Bank77 [4], CLINC150 [12], Mtop [13], and Massive [7] for our experiments. We adopt the same example sets: 3,080 examples with 77 intents from Bank77, 4,500 examples with 150 intents from CLINC150, 4,386 examples with 102 intents from Mtop, and 2,974 examples with 59 intents from Massive. Bank77 is single-domain with fine-grained intents, CLINC150 is multi-domain (10 domains), and MTop and MASSIVE are imbalanced datasets. We use all-MiniLM-L6-v2 [20] and Instructor-large [23] as the backbone embedders as they are commonly used in prior work. We use Gemma-2-9b-it [21] and Llama3.1-8B-Instruct [1] as the LLMs.

Baseline Methods KeyphraseCluster [25] and SPILL [16] are directly comparable to LUMI since we do not use the number of clusters K in the representation refinement stage. In contrast, this prior knowledge of the number of clusters is required in ClusterLLM [26] and LLMedgeRefine [6] to refine the embeddings. For IDAS [5], we reproduce its known K and unknown K scenarios.

Evaluation Following most prior work [5, 16, 24, 26, 27], we apply K-means, the most widely used and standard approach for deriving clusters from the constructed embeddings. For evaluation purposes against the labeled test set, the number of clusters is set to the number of ground-truth labels, K , as in all prior work, to ensure a fair comparison. After obtaining the clusters, we evaluate them using standard clustering metrics: normalized mutual information (NMI) and clustering accuracy (Acc) [8, 10, 17, 19].

5.2 Main results

Table 1 shows the comparison between our approach and other SOTA baselines by embedders and LLMs. First, it shows that LUMI is consistently better with both backbone embedders than all other approaches. This implies two points: First, LUMI is more useful and practical than the prior methods IDAS, LLMedgeRefine, and CLusterLLM, which use the cluster K information in the embedding refinement; omitting the ground-truth K in IDAS leads to a drop in performance. Second, we obtain a stronger result than methods using GPT3.5 (by 10-13 percentage points on ACC) and GPT4 (by 0-3 percentage points on ACC), which means that LUMI reduces the reliance on large-size models. Analyzing results by dataset, our approach outperforms SOTA baselines on three of the four datasets, with the exception of Bank77, where performance fluctuates relative to other SOTA baselines (see Section 5.3.3 for discussion).

5.3 Analysis

5.3.1 Ablation study. Figure 2 shows the cumulative effect of adding each method component. Overall, pooling text embeddings with averaged pseudo-label embedding leads to higher ACC and NMI compared to pretrained embeddings. Applying text-level pooling with Jaccard similarity weights further improves performance on three out of the four datasets.

Table 1: Results on 4 benchmark sets (averages over 5 runs). **Bold** = best within each embedder. Underlined = best within each LLM. * = statistically significant vs. strongest baseline (IDAS†) on the same dataset within each LLM and embedder. An † indicates that the method incorporates known K in the embedding refinement. IntentGPT, LLMEdgeRefine, KeyphraseClust. and ClusterLLM results cited from prior work.

	Bank 77		Cling150		Mtop		Massive		Average	
	NMI	Acc	NMI	Acc	NMI	Acc	NMI	Acc	NMI	Acc
Backbone embedder: MiniLM-L6-v2										
GPT-4 + IntentGPT	81.42	64.22	94.35	83.20	-	-	-	-	-	-
GPT3.5 + IntentGPT	76.58	54.82	90.04	73.68	-	-	-	-	-	-
LLama + SPILL	78.95	63.56	88.71	75.67	67.73	35.88	69.93	54.84	76.33	57.49
+ IDAS	81.64	63.69	91.81	78.81	72.12	40.74	73.13	55.58	79.67	59.70
+ IDAS†	80.82	62.48	91.64	81.72	71.52	39.74	74.06	59.77	79.51	60.93
+ LUMI (ours)	82.78*	67.33*	93.21*	83.13*	73.86*	46.49*	74.94*	61.77*	81.20	64.68
Gemma + SPILL	81.92	66.22	90.78	77.76	70.48	37.95	72.59	58.05	78.94	60.00
+ IDAS	83.01	65.96	92.82	80.05	70.46	35.55	75.67	59.52	80.49	60.27
+ IDAS†	84.88	72.77	93.41	85.52	70.88	36.46	75.93	57.75	81.27	63.13
+ LUMI (ours)	82.59	64.54	95.05*	86.93*	76.88*	49.10*	77.66*	64.18*	83.04	66.19
Backbone embedder: Instructor-large										
GPT3.5 + KeyphraseClust.	82.4	65.3	92.6	79.4	-	-	-	-	-	-
+ ClusterLLM†	85.15	71.20	94.00	83.80	73.83	35.04	<u>77.64</u>	60.69	82.66	62.68
+ LLMEdgeRefine†	-	-	<u>94.86</u>	<u>86.77</u>	<u>72.92</u>	<u>46.00</u>	<u>76.66</u>	<u>63.42</u>	-	-
LLama + SPILL	83.59	69.93	92.90	84.59	71.11	35.26	75.56	60.06	80.79	62.46
+ IDAS	79.94	60.49	93.06	82.38	73.68	39.74	75.76	59.34	80.61	60.49
+ IDAS†	81.79	67.45	92.12	81.71	71.69	37.25	76.88	<u>64.38</u>	80.62	62.70
+ LUMI (ours)	83.72*	67.12	94.56*	85.86*	75.30*	46.05*	77.33*	<u>63.91</u>	82.73	65.73
Gemma + SPILL	<u>85.01</u>	71.05	93.77	85.14	72.65	37.11	77.62	62.42	82.26	63.93
+ IDAS	83.69	66.18	93.74	81.04	73.26	37.18	77.39	61.69	82.02	61.52
+ IDAS†	84.45	71.56	95.00	88.82	72.24	38.28	78.04	61.89	82.43	65.14
+ LUMI (ours)	83.54	65.72	95.72*	87.31	79.11*	53.95*	78.88*	65.32*	84.31	68.08

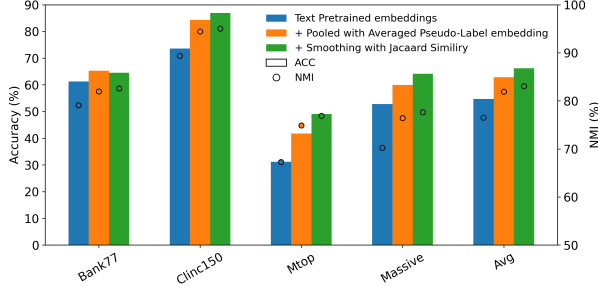


Figure 2: Cumulative contributions of method components: label-level and text-level pooling. Embedder: MiniLM. LLM: Gemma. Averages over 5 runs.

5.3.2 Variation of Candidate Count b . We set the top-10 budget ($b = 10$) for all experiments. To analyze the effect of candidate size, we vary $b \in \{6, 8, 10, 12, 14\}$ using MiniLM-L6-v2 and Gemma. All obtained NMI scores are between 82.4 and 83.7 and all Acc scores between 65.3 and 67.2, indicating that LUMI is relatively robust to variations in the candidate count.

5.3.3 Error Analysis. We hypothesize that Bank77’s lower performance is due to pseudo-labels providing minimal denoising. To verify this, we analyze the results in detail (with MiniLM as embedder and Gemma as LLM). We find that, on average, each pseudo-label in Bank77 includes 1.70 ground truth clusters, while in CLINC150 it includes only 1.19, indicating that the pseudo-labels in Bank77

are noisier. Figure 2 also shows that the increase margin from averaged pseudo-label embeddings is the lowest compared to the others datasets. We manually checked the pseudo-labels and real labels. We found that the Banking intent is very fine-grained, e.g. “I topped up my card but the money disappeared” and “My card was topped this morning but I can’t see the funds. Why didn’t it complete?” have two different intents: ‘automatic top up’ and ‘pending top up’. LUMI’s pseudo-label ‘card top up issue’ covers both. This suggests that few-shot examples may be needed for finer-grained intents.

6 Conclusion

We propose a novel method, LUMI, to improve text representations for short-text clustering using multiple pseudo-labels as guidance. LUMI does not rely on the impractical assumption of knowing the number of clusters K and requires no labeled data to refine embeddings. We provide a theoretical analysis to explain why using multiple pseudo-labels improves performance. Our results show that LUMI overall outperform than SOTA methods across different LLMs and embedders. This indicates that our method is an effective training-free approach for unsupervised intent clustering. In the future, we plan to explore how our approach can be further improved on finer-grained datasets such as Bank77.

References

- [1] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [2] Daria Alexander and Arjen P de Vries. 2025. In a Few Words: Comparing Weak Supervision and LLMs for Short Query Intent Classification. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2977–2981.
- [3] Martin Borčín and Joemon M Jose. 2024. Optimizing BERTopic: Analysis and reproducibility study of parameter influences on topic modeling. In *European Conference on Information Retrieval*. Springer, 147–160.
- [4] Iñigo Casanueva, Tadas Temcinas, Daniela Gerz, Matthew Henderson, and Ivan Vulic. 2020. Efficient Intent Detection with Dual Sentence Encoders. In *Proceedings of the 2nd Workshop on NLP for ConvAI - ACL 2020*. <https://arxiv.org/abs/2003.04807> Data available at <https://github.com/PolyAI-LDN/task-specific-datasets>.
- [5] Maarten De Raedt, Frédéric Godin, Thomas Demeester, and Chris Develder. 2023. IDAS: Intent Discovery with Abstractive Summarization. In *Proceedings of the 5th Workshop on NLP for Conversational AI (NLP4ConvAI 2023)*. 71–88.
- [6] Zijin Feng, Luyang Lin, Lingzhi Wang, Hong Cheng, and Kam-Fai Wong. 2024. LLMEdgeRefine: Enhancing text clustering with LLM-based boundary point refinement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 18455–18462.
- [7] Jack Fitzgerald, Christopher Hench, Charith Peris, Scott Mackie, Kay Rottmann, Ana Sanchez, Aaron Nash, Liam Urbach, Vishesh Kakarala, Richa Singh, et al. 2023. MASSIVE: A 1M-Example Multilingual Natural Language Understanding Dataset with 51 Typologically-Diverse Languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4277–4302.
- [8] James Gung, Raphael Shu, Emily Moeng, Wesley Rose, Salvatore Romeo, Arshit Gupta, Yassine Benajiba, Saab Mansour, and Yi Zhang. 2023. Intent Induction from Conversations for Task-Oriented Dialogue Track at DSTC 11. In *Proceedings of The Eleventh Dialog System Technology Challenge*. 242–259.
- [9] Yuan Hong, Jaideep Vaidya, Haibing Lu, and Wen Ming Liu. 2016. Accurate and efficient query clustering via top ranked search results. In *Web Intelligence*, Vol. 14. SAGE Publications Sage UK: London, England, 119–138.
- [10] Peihao Huang, Yan Huang, Wei Wang, and Liang Wang. 2014. Deep embedding network for clustering. In *2014 22nd International conference on pattern recognition*. IEEE, 1532–1537.
- [11] Markus Koskela, Petri Luukkonen, Tuukka Ruotsalo, Mats Sjöberg, and Patrik Florén. 2018. Proactive information retrieval by capturing search intent from primary task context. *ACM Transactions on Interactive Intelligent Systems (TiiS)* 8, 3 (2018), 1–25.
- [12] Stefan Larson, Anish Mahendran, Joseph J Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K Kummerfeld, Kevin Leach, Michael A Laurenzano, Lingjia Tang, et al. 2019. An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 1311–1316.
- [13] Haoran Li, Abhinav Arora, Shuohui Chen, Anchit Gupta, Sonal Gupta, and Yashar Mehdad. 2020. MTOP: A comprehensive multilingual task-oriented semantic parsing benchmark. *arXiv preprint arXiv:2008.09335* (2020).
- [14] Jinggui Liang, Lizi Liao, Hao Fei, Bobo Li, and Jing Jiang. 2024. Actively learn from llms with uncertainty propagation for generalized category discovery. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 7838–7851.
- [15] Jinggui Liang, Yuxia Wu, Yuan Fang, Hao Fei, and Lizi Liao. 2024. A Survey of Ontology Expansion for Conversational Understanding. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 18111–18127.
- [16] I-Fan Lin, Faegheh Hasibi, and Suzan Verberne. 2025. SPILL: Domain-Adaptive Intent Clustering based on Selection and Pooling with Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 15723–15737. doi:10.18653/v1/2025.findings-acl.812
- [17] Marina Meilă. 2007. Comparing clusterings—an information based distance. *Journal of multivariate analysis* 98, 5 (2007), 873–895.
- [18] Chen Qu, Liu Yang, W Bruce Croft, Yongfeng Zhang, Johanne R Trippas, and Minghui Qiu. 2019. User intent prediction in information-seeking conversations. In *Proceedings of the 2019 Conference on Human Information Interaction and Retrieval*. 25–33.
- [19] William M Rand. 1971. Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical association* 66, 336 (1971), 846–850.
- [20] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. <https://arxiv.org/abs/1908.10084>
- [21] Morgane Rivi re, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, L onard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ram , Johan Ferret, et al. 2024. Gemma 2: Improving Open Language Models at a Practical Size. *CoRR* (2024).
- [22] Juan A Rodriguez, Nicholas Botzer, David Vazquez, Christopher Pal, Marco Pedersoli, and Issam H Laradji. 2024. IntentGPT: Few-shot Intent Discovery with Large Language Models. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- [23] Hongjin Su, Weijia Shi, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A Smith, Luke Zettlemoyer, and Tao Yu. 2023. One Embedder, Any Task: Instruction-Finetuned Text Embeddings. In *Findings of the Association for Computational Linguistics: ACL 2023*. 1102–1121.
- [24] Vijay Viswanathan, Kiril Gashteovski, Kiril Gashteovski, Carolin Lawrence, Tongshuang Wu, and Graham Neubig. 2024. Large Language Models Enable Few-Shot Clustering. *Transactions of the Association for Computational Linguistics* 12 (2024), 321–333. doi:10.1162/tacl_a_00648
- [25] Vijay Viswanathan, Kiril Gashteovski, Carolin Lawrence, Tongshuang Wu, and Graham Neubig. 2024. Large Language Models Enable Few-Shot Clustering. *Transactions of the Association for Computational Linguistics* 11 (2024), 321–333.
- [26] Yuwei Zhang, Zihan Wang, and Jingbo Shang. 2023. ClusterLLM: Large Language Models as a Guide for Text Clustering. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 13903–13920.
- [27] Henry Peng Zou, Sift Singh, Yi Nian, Jianfeng He, Jason Cai, Saab Mansour, and Hang Su. 2025. GLEAN: Generalized Category Discovery with Diverse and Quality-Enhanced LLM Feedback. *CoRR* (2025).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009